

Soutenance LO07 – BlaBlaCar 2026

Partie de Moss'Ab : les couches **Contrôleur** et **Modèle**

Moss'Ab Mirande-Ney (binôme : Pierre Bheidi)

1 Ce que je présente

Je suis responsable des couches **Contrôleur** et **Modèle** : le cœur de l'application. Cela comprend le point d'entrée unique, le routeur, la logique métier (authentification, paiements, réservations) et tout l'accès à la base de données via PDO et des requêtes préparées.

2 Texte à dire pendant la démo

2.1 1. Le Front Controller (index.php)

« Toutes les requêtes passent par un seul fichier, `index.php`. Il démarre la session, charge la configuration, les modèles et le routeur, puis délègue à celui-ci. Au tout premier accès d'une session, on remet `login_id` à -1 pour garantir que personne n'est connecté au démarrage, comme demandé dans le sujet. »

```
if (!isset($_SESSION['app_demarree'])) {  
    $_SESSION['app_demarree'] = true;  
    $_SESSION['login_id'] = -1;    // personne connecte au démarrage  
}  
Router::dispatch($action);
```

2.2 2. Le routeur déclaratif

« Le routeur est une simple table de correspondance entre une **action** (dans l'URL) et un couple [**Contrôleur**, **méthode**]. Pour ajouter une fonctionnalité, on ajoute juste une ligne. C'est lisible et facile à maintenir. »

```
'admin_utilisateurs' => ['AdminController', 'listeUtilisateurs'], // A1  
'cond_cloturer'      => ['ConducteurController', 'cloturerTrajet'], // C5
```

2.3 3. Le contrôleur de base

« Tous les contrôleurs héritent d'une classe **Controller** qui fournit : `render()` (affiche une vue dans le gabarit), `redirect()`, `exigerRole()` (le contrôle d'accès), et un système de messages flash. »

```
protected function exigierRole(array $rolesAutorises): array {  
    $user = $this->utilisateurConnecte();  
    if ($user === null || !in_array($user['role'], $rolesAutorises, true)) {  
        $this->redirect('login');    // acces refuse  
    }  
    return $user;  
}
```

2.4 4. L'authentification (F1 / F2)

« Le `AuthController` vérifie le couple login / mot de passe via le modèle. Si c'est bon, on stocke l'id en session et on redirige vers le menu du rôle. La déconnexion remet `login_id` à -1. »

2.5 5. La logique métier la plus intéressante : la clôture (C5)

« Quand un conducteur clôture un trajet, deux choses se passent : le trajet passe en statut passif, ET les paiements sont réglés. Pour chaque réservation, le passager est débité du prix et le conducteur crédité d'autant. On vérifie d'abord que le trajet appartient bien au conducteur connecté. »

```
foreach ($reservations as $resa) {  
    $utilisateurModel->ajusterSolde($resa['passager_id'], -$prix); // debit  
    $utilisateurModel->ajusterSolde($user['id'], $prix);           // credit  
}  
$trajetModel->cloturer($trajetId); // statut -> passif
```

2.6 6. Les modèles et la couche d'accès

« Tous les modèles héritent d'une classe abstraite `Model` qui centralise la connexion PDO et des utilitaires : `fetchAll`, `fetchOne`, `execute`, `nextId`. **Seule cette couche parle à la base** : ni les contrôleurs ni les vues n'écrivent de SQL brut directement. »

2.7 7. La connexion PDO (singleton)

« La connexion à MySQL est un singleton : une seule connexion est créée et partagée pour toute la requête HTTP. PDO est configuré pour lever des exceptions et on utilise des requêtes préparées partout. »

2.8 8. Sécurité : requêtes préparées

« Toutes les requêtes utilisent des paramètres liés (?), ce qui protège contre les injections SQL : les valeurs saisies par l'utilisateur ne sont jamais concaténées dans le SQL. »

```
$this->fetchOne(  
    'SELECT * FROM utilisateur WHERE login = ? AND password = ?',  
    [$login, $password]  
);
```

3 Ce que je montre à l'écran

1. Connexion en administrateur : ajout d'un conducteur, d'un véhicule, d'une ville.
2. Connexion en conducteur : création d'un trajet, puis **clôture** en montrant les soldes qui changent (passager débité, conducteur crédité).
3. Connexion en passager : réservation d'un trajet actif.
4. Menu Examineur : E1 (superglobales) et E2 (10 réservations aléatoires).

4 Questions potentielles du jury (et réponses)

Q : Qu'est-ce qu'un Front Controller ?

R : Un point d'entrée unique par lequel passent toutes les requêtes (`index.php`). Il centralise l'initialisation (session, config, routeur) et évite de dupliquer ce code dans chaque page.

Q : Comment fonctionne votre routeur ?

R : Il contient un tableau associatif `action → [Contrôleur, méthode]`. La méthode `dispatch()` lit l'action dans l'URL, instancie le bon contrôleur et appelle la méthode. Si l'action est inconnue, on renvoie l'accueil.

Q : Comment gérez-vous l'authentification et les rôles ?

R : Après connexion, on stocke l'id de l'utilisateur en session. Chaque contrôleur protégé appelle `exigerRole()` qui vérifie que l'utilisateur connecté a bien le rôle requis, sinon il est redirigé vers le login.

Q : Pourquoi des requêtes préparées ?

R : Pour empêcher les injections SQL. Les valeurs sont passées séparément de la requête, donc un utilisateur ne peut pas injecter de code SQL via un champ de formulaire.

Q : Pourquoi un singleton pour la connexion PDO ?

R : Pour n'ouvrir qu'une seule connexion par requête HTTP et la partager entre tous les modèles, ce qui évite de gaspiller des connexions.

Q : Décrivez précisément ce qui se passe à la clôture d'un trajet.

R : On vérifie que le trajet appartient au conducteur et qu'il est actif. Puis, pour chaque réservation, on débite le passager du prix et on crédite le conducteur. Enfin on passe le trajet en statut passif pour bloquer les nouvelles réservations.

Q : Et si un passager a réservé deux fois le même trajet ?

R : Il est facturé deux fois : on parcourt toutes les lignes de réservation, donc deux réservations entraînent deux débits. C'est le comportement attendu par le sujet.

Q : Pourquoi calculer l'id avec `nextId` au lieu d'`AUTO_INCREMENT` ?

R : Le jeu de données fourni insère les id manuellement, sans `AUTO_INCREMENT`. Pour rester cohérent, on prend `MAX(id)+1` lors d'un ajout.

Q : Différence entre `fetchAll` et `fetchOne` ?

R : `fetchAll` renvoie toutes les lignes (une liste), `fetchOne` renvoie la première ligne seule (ou null). On choisit selon qu'on attend une liste ou un seul enregistrement.

Q : Comment le modèle communique-t-il avec la vue ?

R : Il ne communique pas directement. Le contrôleur récupère les données du modèle, puis les transmet à la vue via `render()`. Le modèle ignore tout de l'affichage.

Q : Comment empêchez-vous un conducteur de clôturer le trajet d'un autre ?

R : Avant toute action, on récupère le trajet et on vérifie que son `conducteur_id` est bien celui de l'utilisateur connecté. Sinon l'action est refusée.

Q : Avez-vous utilisé une transaction pour les paiements ?

R : Dans cette version, les mises à jour de solde sont successives. Une amélioration serait de les encapsuler dans une transaction PDO (`beginTransaction` / `commit`) pour garantir l'atomicité – c'est un axe d'évolution que l'on peut mentionner.

Q : Comment l'application s'adapte-t-elle au serveur (local / dev-isi) ?

R : Le fichier de configuration détecte l'environnement (variables d'env, ou présence de `dev-isi.utt.fr` dans l'URL) et choisit automatiquement les bons identifiants de base de données.